

Lien du Github :
<https://github.com/Sidnehh/ProjetBDD2ArtConnect>

Lien de la présentation Canva :
<https://canva.link/rc61viiake03hbt>

Projet : ArtConnect

Yoann SEURAT, Sidney SANGLIER, Arthur SAVATIER, Kevin SIVAHARAN

Table des matières

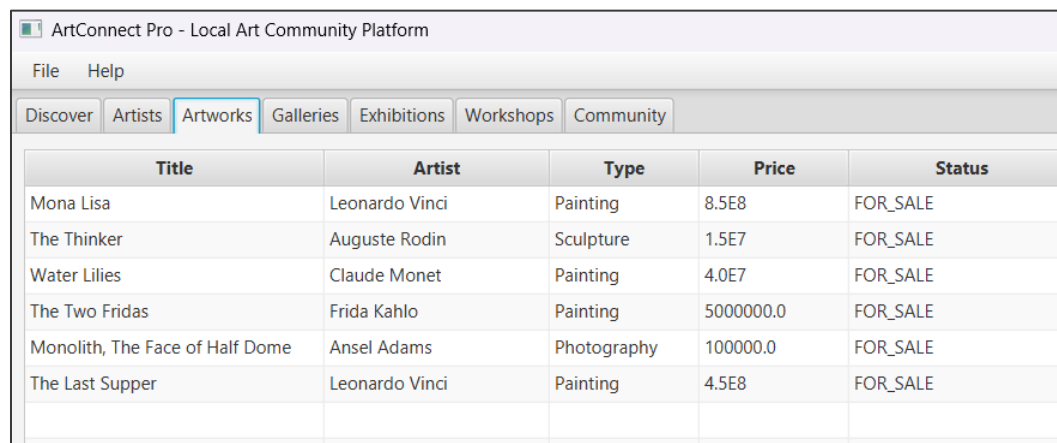
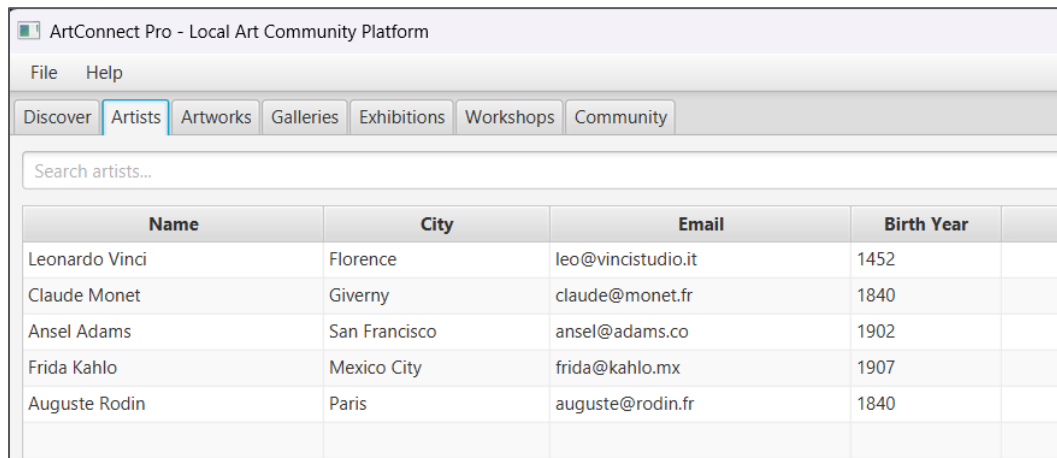
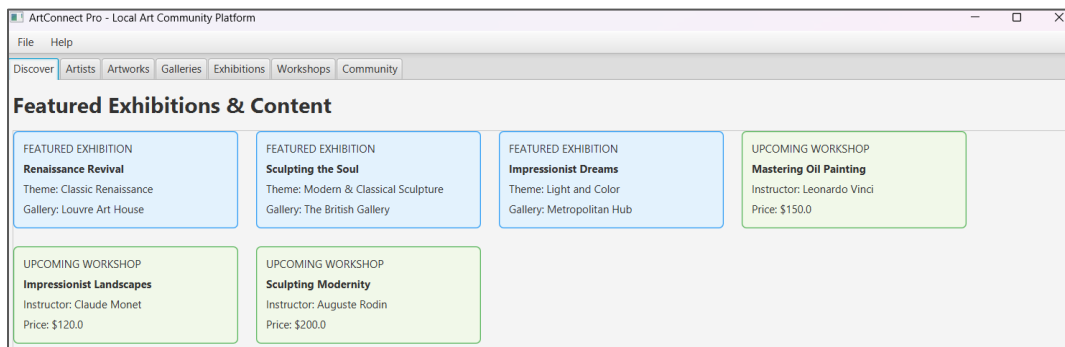
Projet : ArtConnect.....	1
Table des matières.....	1
Livrable 1 : Comprendre et définir le périmètre fonctionnel.....	2
1. Exploration de l'application fournie	2
2. Analyse fonctionnelle	4
3. Conception statique	5
Livrable 2 : Modélisation de la base.....	6
1. Identifier les entités et les relations	6
2. Créer un modèle conceptuel des données (MCD).....	7
3. Créer un modèle logique des données (MLD)	7
4. Créer le schéma de la base de données	8
Livrable 3 : Implémentation et fonctionnalités avancées	10
1. Création et remplissage de la base.....	10
2. Vues, indexes et droits d'accès	10
3. Déclencheurs et programmes stockés	11
4. Transactions	12
Livrable 4 : Intégration de la base dans l'application Java	13
1. DAO (Data Access Objects).....	13
2. Services	13
3. Connexion à la base.....	13
4. Adaptation de l'UI	14

Livrable 1 : Comprendre et définir le périmètre fonctionnel

1. Exploration de l'application fournie

Identification des écrans principaux

Ecrans principaux : Discover, Artists, Artworks, Galleries, Exhibitions, Workshops, Community



ArtConnect Pro - Local Art Community Platform						
File Help						
Discover	Artists	Artworks	Galleries	Exhibitions	Workshops	Community
Louvre Art House - Rue de Rivoli, Paris (4.9/5.0)						
The British Gallery - Great Russell St, London (4.7/5.0)						
Metropolitan Hub - 1000 5th Ave, New York (4.8/5.0)						

ArtConnect Pro - Local Art Community Platform						
File Help						
Discover	Artists	Artworks	Galleries	Exhibitions	Workshops	Community
Title	Gallery	Start Date	Theme			
Renaissance Revival	Louvre Art House	2026-02-28	Classic Renaissance			
Sculpting the Soul	The British Gallery	2026-03-15	Modern & Classical Sculpture			
Impressionist Dreams	Metropolitan Hub	2026-01-30	Light and Color			

ArtConnect Pro - Local Art Community Platform						
File Help						
Discover	Artists	Artworks	Galleries	Exhibitions	Workshops	Community
Title	Instructor	Date	Price	Level		
Mastering Oil Painting	Leonardo Vinci	2026-04-04T11:30:41.600...	150.0	Intermediate		
Impressionist Landscapes	Claude Monet	2026-04-09T11:30:41.600...	120.0	Beginner		
Sculpting Modernity	Auguste Rodin	2026-04-14T11:30:41.601...	200.0	Advanced		

ArtConnect Pro - Local Art Community Platform						
File Help						
Discover	Artists	Artworks	Galleries	Exhibitions	Workshops	Community
Name	Email	City				
Alice Wonderland	alice@art.com	Paris				
Bob Ross	bob@happytrees.com	London				
Charlie Brown	charlie@peanuts.com	New York				

2. Analyse fonctionnelle

Liste des fonctionnalités principales que vous de l'interface

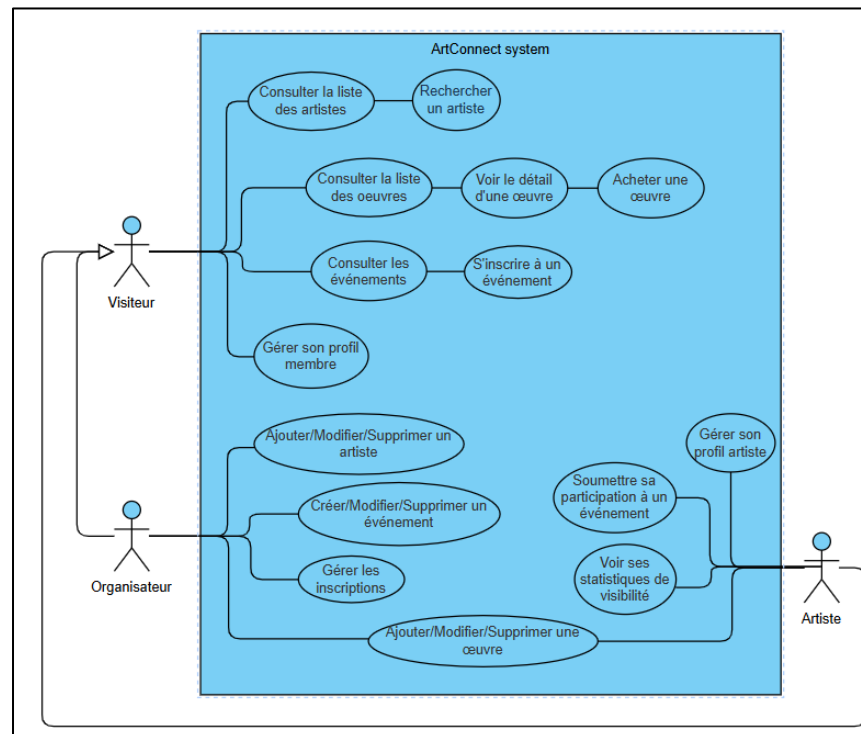
L'utilisateur a accès à « File » et « Help » permettant, à l'heure actuelle, de quitter l'application et d'obtenir des renseignements. Il peut également accéder à plusieurs onglets montrant des informations spécifiques selon ce qu'il sélectionne (Artistes, Galleries, Expositions, etc...). Enfin, il peut avoir un aperçu rapide des dernières expositions et workshop depuis l'onglet découvertes (« Discover ») servant d'accueil.

Identification de rôles ou profils d'utilisation

On peut imaginer ces différents profils d'utilisation :

- Artiste
- Visiteur / Client
- Organisateur

Diagramme de cas d'utilisation (UML) des fonctionnalités cibles d'ArtConnect



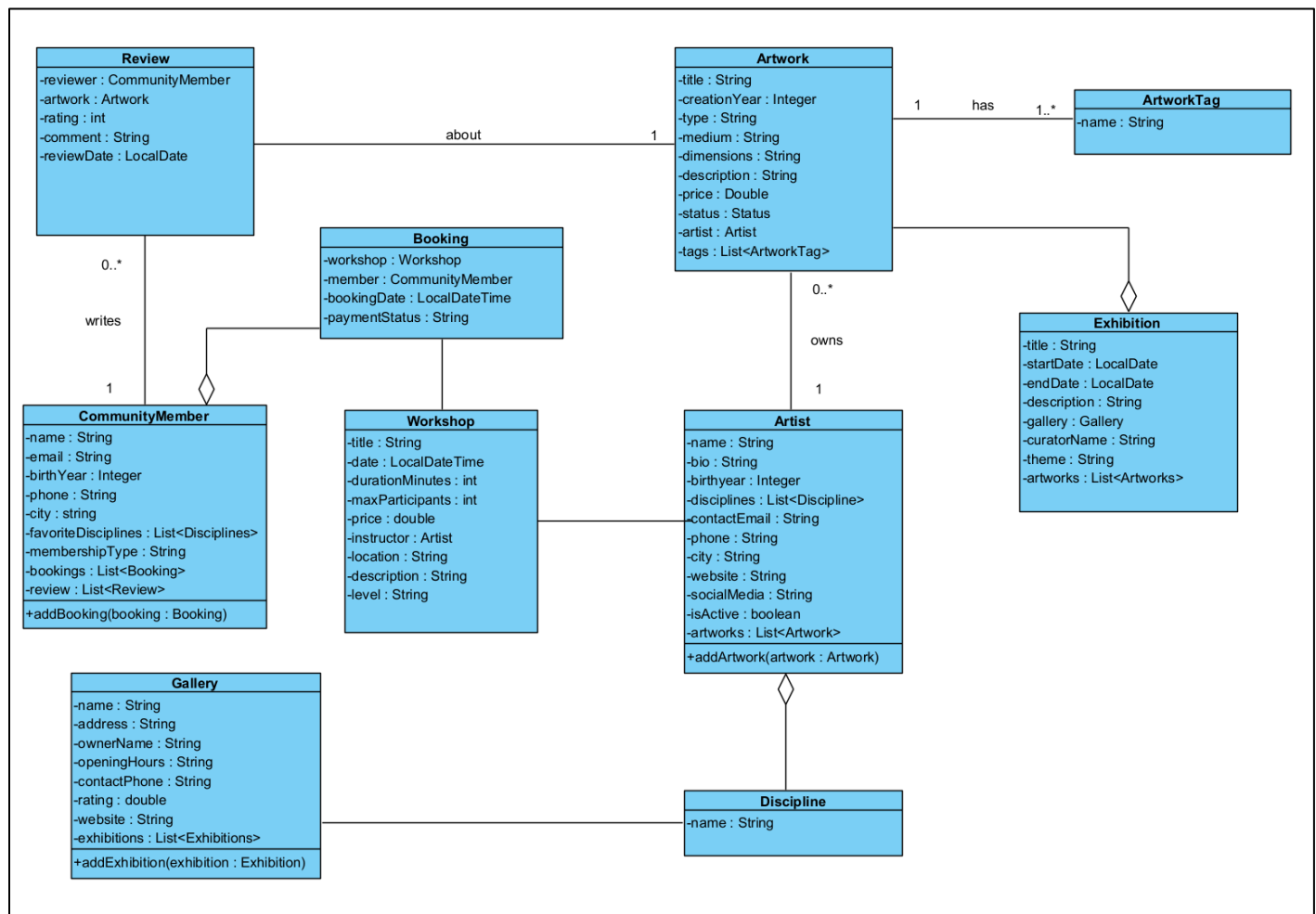
Ajoutez une légende indiquant quelles fonctionnalités sont déjà présentes dans l'interface et lesquelles sont envisagées pour plus tard.

Les seules fonctionnalités déjà présentes sont : Consulter la liste des artistes, des œuvres, des musées, des événements, des workshops, et des communautés.

On peut donc envisager pour plus tard toutes les autres fonctions d'ajout/modification/suppression, de gestion de compte et d'accès aux fonctionnalités.

3. Conception statique

Réaliser un (ou plusieurs) diagramme(s) de classe pour le code. Représenter de préférence les classes du modèle dans un diagramme séparé.



Livrable 2 : Modélisation de la base

1. Identifier les entités et les relations

À partir de l'interface et du besoin métier, identifiez les différentes entités ainsi que leurs attributs.

On a les entités : Artists, Artworks, Galleries, Exhibitions, Workshops, Community

- Pour un artiste : Name, City, Email, Birth year
- Pour une oeuvre : Title, Artist, Type, Price, Status
- Pour un musée : Name, Street name, City, Rating
- Pour une exhibition : Title, Gallery, Start date, Theme
- Pour un workshop : Title, Instructor, Date, Price, Level
- Pour un membre d'une communauté : Name, Email, City

Définir les dépendances fonctionnelles entre les différents attributs.

Artists : Email -> Name, City, Birth year

Artwork : {Title, Artist} -> Type, Price, Status

Galleries : Name -> Street Name, City, Rating

Exhibitions : {Title, Start Date} -> Gallery, Theme

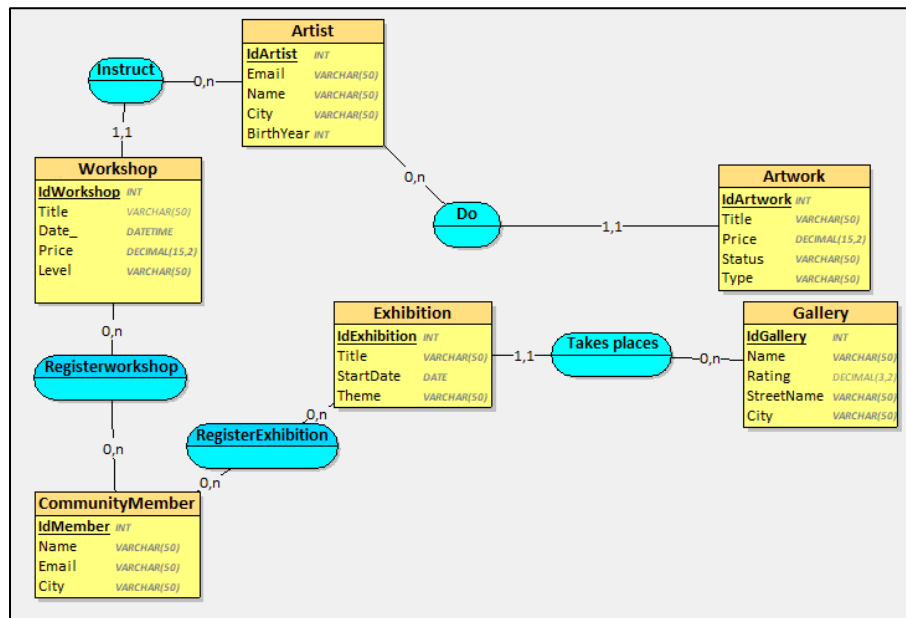
Workshops : {Title, Date, Instructor} -> Price, Level

Community : Email -> Name, City

Réfléchir aux identifiants à définir pour chaque entité. Vous pouvez choisir un (ou plusieurs) attribut(s) déjà existants dans la classe associée, ou alors en définir un nouveau.

On préfère mettre un nouvel attribut Id[NomDeL'Entité] pour toutes les entités, ça nous facilitera la tâche plus tard.

2. Créer un modèle conceptuel des données (MCD)



3. Créer un modèle logique des données (MLD)

Transformer le MCD en MLD en ajoutant les clés primaires et les clés étrangère

Artist = (IdArtist INT, Email VARCHAR(50), Name VARCHAR(50), City VARCHAR(50), BirthYear INT);

Artwork = (IdArtwork INT, Title VARCHAR(50), Price DECIMAL(15,2), Status VARCHAR(50), Type VARCHAR(50), #IdArtist);

Gallery = (IdGallery INT, Name VARCHAR(50), Rating DECIMAL(3,2), StreetName VARCHAR(50), City VARCHAR(50));

Exhibition = (IdExhibition INT, Title VARCHAR(50), StartDate DATE, Theme VARCHAR(50), #IdGallery);

CommunityMember = (IdMember INT, Name VARCHAR(50), Email VARCHAR(50), City VARCHAR(50));

Workshop = (IdWorkshop INT, Title VARCHAR(50), Date_ DATETIME, Price DECIMAL(15,2), Level VARCHAR(50), #IdArtist);

RegisterExhibition = (#IdExhibition, #IdMember);

Registerworkshop = (#IdMember, #IdWorkshop);

La base de données est déjà en 3FN (Tous les attributs sont atomiques, tout attribut non-clé dépend de toute la clé primaire, tout attribut non-clé ne dépend que de la clé primaire).

4. Créer le schéma de la base de données

Écrivez des scripts SQL pour créer les tables, définir les contraintes (par exemple, clés primaires, clés étrangères, NOT NULL) et établir les relations.

```
CREATE DATABASE IF NOT EXISTS ArtConnect;
USE ArtConnect;

CREATE TABLE IF NOT EXISTS Artist(
  IdArtist INT,
  Email VARCHAR(50) NOT NULL,
  Name VARCHAR(50) NOT NULL,
  City VARCHAR(50) NOT NULL,
  BirthYear INT NOT NULL,
  PRIMARY KEY(IdArtist)
);

CREATE TABLE IF NOT EXISTS Artwork(
  IdArtwork INT,
  Title VARCHAR(50) NOT NULL,
  Price DECIMAL(15,2) NOT NULL,
  Status VARCHAR(50) NOT NULL,
  Type VARCHAR(50) NOT NULL,
  IdArtist INT NOT NULL,
  PRIMARY KEY(IdArtwork),
  FOREIGN KEY(IdArtist) REFERENCES Artist(IdArtist)
);

CREATE TABLE IF NOT EXISTS Gallery(
  IdGallery INT,
  Name VARCHAR(50) NOT NULL,
  Rating DECIMAL(3,2),
  StreetName VARCHAR(50) NOT NULL,
  City VARCHAR(50) NOT NULL,
  PRIMARY KEY(IdGallery)
);

CREATE TABLE IF NOT EXISTS Exhibition(
  IdExhibition INT,
  Title VARCHAR(50) NOT NULL,
  StartDate DATE NOT NULL,
  Theme VARCHAR(50) NOT NULL,
  IdGallery INT NOT NULL,
  PRIMARY KEY(IdExhibition),
  FOREIGN KEY(IdGallery) REFERENCES Gallery(IdGallery)
);

CREATE TABLE IF NOT EXISTS CommunityMember(
  IdMember INT,
  Name VARCHAR(50) NOT NULL,
  Email VARCHAR(50) NOT NULL,
  City VARCHAR(50) NOT NULL,
  PRIMARY KEY(IdMember)
);

CREATE TABLE IF NOT EXISTS Workshop(
  IdWorkshop INT,
  Title VARCHAR(50),
  Date_ DATETIME NOT NULL,
  Price DECIMAL(15,2) NOT NULL,
  Level VARCHAR(50) NOT NULL,
  IdArtist INT NOT NULL,
  PRIMARY KEY(IdWorkshop),
  FOREIGN KEY(IdArtist) REFERENCES Artist(IdArtist)
);

CREATE TABLE IF NOT EXISTS RegisterExhibition(
  IdExhibition INT,
  IdMember INT,
  PRIMARY KEY(IdExhibition, IdMember),
  FOREIGN KEY(IdExhibition) REFERENCES Exhibition(IdExhibition),
  FOREIGN KEY(IdMember) REFERENCES CommunityMember(IdMember)
```



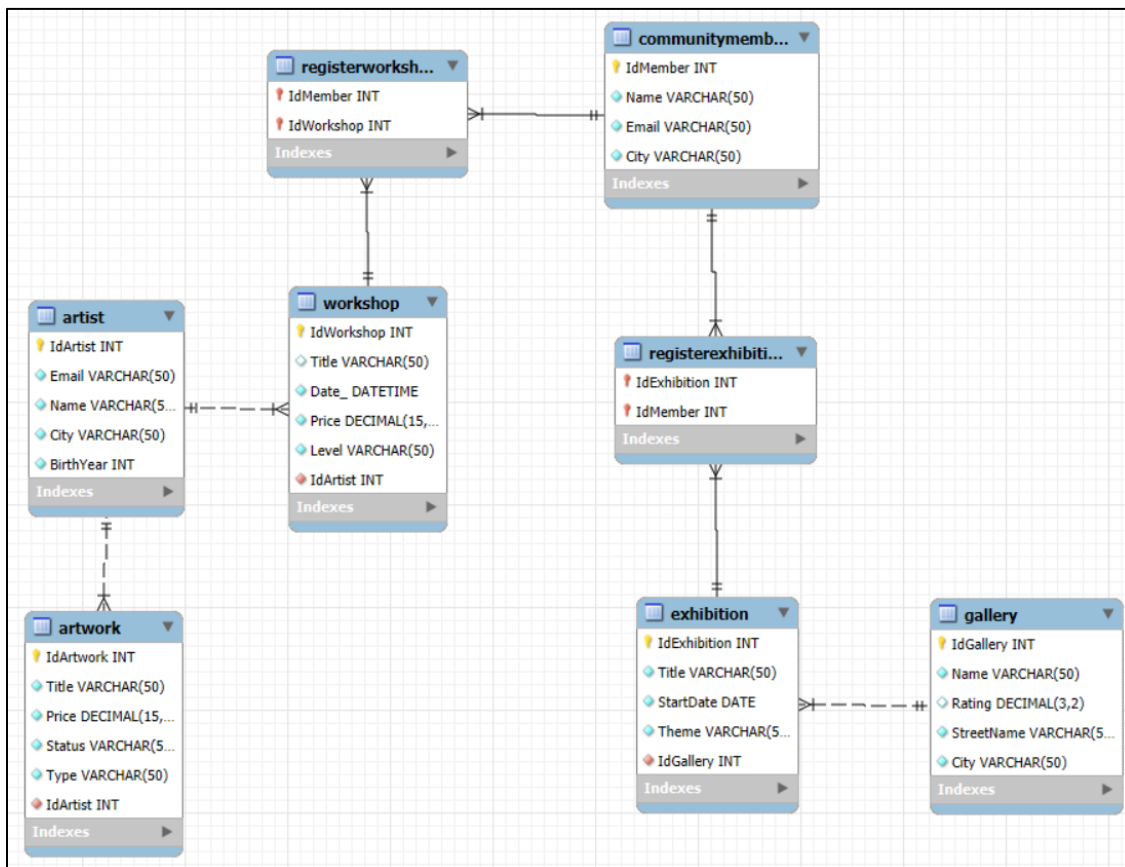
```

);

CREATE TABLE IF NOT EXISTS Registerworkshop (
  IdMember INT,
  IdWorkshop INT,
  PRIMARY KEY (IdMember, IdWorkshop),
  FOREIGN KEY (IdMember) REFERENCES CommunityMember (IdMember),
  FOREIGN KEY (IdWorkshop) REFERENCES Workshop (IdWorkshop)
);

```

Après avoir exécuté ces commandes et effectué du reverse engineering, on obtient :



Livrable 3 : Implémentation et fonctionnalités avancées

1. Création et remplissage de la base

Créez un script d'insertion de données d'exemple (artistes, œuvres, événements, membres...) permettant de montrer des cas intéressants (plusieurs artistes, plusieurs événements, participations croisées, etc.)

Nous avons utilisé Claude pour générer les requêtes d'insertion de données, avec le prompt ci-dessous et en lui donnant les pdf des Livrables et du Projet.

« voici un projet de bases de données avec ses consignes, notre livrable et la création de la base et des tables en sql. fais-nous les requêtes d'insertion des données pour toutes les tables : un assez grand nombre de données, chacun unique et intéressant, avec assez de cas à analyser lors de la création des vues/index/procédures après. »

Les requêtes SQL qu'il nous a retourné sont dans le fichier correspondant sur GitHub.

2. Vues, indexes et droits d'accès

Créez les vues de votre choix. Chaque vue doit avoir un objectif clair: sécurité, simplification de requête, masquage de certains attributs, etc.

Pour les vues elles correspondront avec l'utilisation de l'application dans 3 rôles différents. On distingue l'utilisateur, l'artiste et l'organisateur.

Voici les vues retenues pour les artistes :

- `view_artist_portfolio` : Permet à un artiste de consulter ses œuvres et leur statut de vente
- `view_artist_workshop_members` : Donne à l'artiste la liste des participants à ses workshops

Voici les vues retenues pour les utilisateurs :

- `view_upcoming_events` : Récupère les workshops et exhibitions à venir pour alimenter la page Discover.
- `view_member_registrations` ; Permet à un membre de voir toutes ses inscriptions (workshops + exhibitions).
- `view_artworks_for_sale` : Montre toutes les œuvres disponibles à la vente

Voici les vues retenues pour les organisateurs :

- `view_admin_members` : Montre les emails et le nombre d'inscriptions de chaque membre.
- `view_activity_summary` : Tableau de bord global par artiste (œuvres, workshops, participants).

Pour ce qui est des indexes on a choisi :

- Artwork(IdArtist), Workshop(IdArtist), Exhibition(IdGallery), Registerworkshop(IdMember), Registerworkshop(IdWorkshop), RegisterExhibition(IdMember), RegisterExhibition(IdExhibition)

Ces attributs sont des clés étrangères utilisées dans les jointures, on accélèrera donc considérablement les requêtes en indexant ces attributs

- Artwork(Status), Workshop(Date_), Exhibition(StartDate)

Ces attributs ci ont besoin d'être indexés car ils sont utilisés dans un WHERE dans les vues

Enfin nous avons créé 3 Users : admin, membre et artiste et leur avons donné les droits d'accès aux vues prévues plus haut.

Les vues, indexes et users sont accessibles dans leur fichier sql respectif.

3. Déclencheurs et programmes stockés

Écrivez des déclencheurs (triggers) pour l'automatisation (par exemple, pour le contrôle des cohérences des dates ou du nombre de places, ou pour l'audit des modification des évènements).

Pour assurer un contrôle des cohérences au sein de la base de données, on met en place les déclencheurs suivants :

- trg_Prevent_Delete_Artist_With_Works : empêche la suppression d'un artiste s'il a encore des œuvres "FOR_SALE"
- trg_Validate_Artwork_Type : vérifie que le type d'une œuvre créée ou modifiée est bien valide parmi Painting, Sculpture, ou Photography, et empêche l'insertion de données dans le cas contraire.
- trg_Validate_Gallery_Rating : vérifie qu'une note assignée à une galerie est bien comprise entre 0 et 5, et empêche l'insertion de données dans le cas contraire.
- trg_Check_Artwork_Price : vérifie que le prix d'une œuvre soit supérieur à 0

Créez des procédures et fonctions stockées pour les opérations courantes (par exemple, créer un évènement en inscrivant automatiquement l'artiste associé, ou retourner le nombre de participants à un évènement).

On crée les procédures suivantes :

- sp_Register_Member_To_Workshop : Gère l'inscription d'un membre à un atelier en vérifiant les contraintes métier (doublons), la procédure :
 1. Vérifie si le membre est déjà inscrit à cet atelier.
 2. Compte le nombre actuel de participants.
 3. Si le membre est déjà inscrit → Retourne un message d'échec.
 4. Sinon → Insère l'enregistrement dans RegisterWorkshop et retourne un message de succès.

- `sp_Create_Workshop` : Gère la création d'un atelier en prenant en compte son identifiant, son nom, son artiste intervenant, son prix, sa date, le niveau requis. La procédure insère une ligne à la table « Workshop », et retourne un message de succès si l'atelier a bien été créé. On considère qu'elle n'inscrit pas automatiquement l'artiste intervenant car notre modèle de base de données ne considère pas un artiste automatiquement comme un membre.
- `sp_Delete_Workshop` : Permet de supprimer un atelier en prenant en compte son identifiant. La procédure supprime la ligne associée de la table « Workshop » et retourne un message de succès si l'atelier a bien été supprimé.
On crée ensuite les programmes stockés suivants :
 - `fn_Count_Exhibition_Participants` : Retourne le nombre exact de participants inscrits à une exposition spécifique. La fonction exécute un `SELECT COUNT(*)` sur la table de liaison `RegisterExhibition` filtré par l'ID de l'exposition.
 - `fn_Count_Workshop_Participants` : Retourne le nombre exact de participants inscrits à un atelier spécifique. La fonction exécute un `SELECT COUNT(*)` sur la table de liaison `RegisterWorkshop` filtré par l'ID de l'exposition.
 - `fn_Count_Artist_Revenue` : Retourne le chiffre d'affaires d'un artiste enregistré en prenant en compte le nombre d'œuvres vendues et les ateliers dans lesquels l'artiste intervient. LA fonction exécute un `SELECT SUM(Price)` sur les tables `Artwork` et `Workshop`.

4. Transactions

Proposez au moins un scénario transactionnel complexe (script SQL) où plusieurs opérations doivent être atomiques (par exemple, inscription d'un membre à plusieurs événements).

On a réfléchi à faire la transaction d'annulation d'atelier par l'organisateur.

Si un organisateur doit annuler un atelier (Workshop) pour une raison majeure (ex: maladie de l'artiste, problème de lieu), cette annulation ne doit pas se limiter à supprimer la ligne de l'atelier. Elle doit garantir deux opérations atomiques pour maintenir la cohérence des données et l'expérience utilisateur :

1. Supprimer toutes les inscriptions liées à cet atelier dans la table `RegisterWorkshop` (pour "rembourser" et libérer les places virtuellement).
2. Supprimer la ligne de l'atelier dans la table `Workshop`.

Livrable 4 : Intégration de la base dans l'application Java

1. DAO (Data Access Objects)

Complétez/implémentez les interfaces DAO fournies dans le projet (par ex. ArtistDao, ArtworkDao, CommunityMemberDao, etc.) dans un package org.project.artconnect.dao.impl

Les interfaces DAO déjà implémentées ont actuellement toutes les méthodes abstraites nécessaires à l'implémentation des classes concrètes du package « persistence ». On décide de les laisser telles quelles.

Implémentez des classes DAO JDBC (par ex. JdbcArtistDao) dans le package org.project.artconnect.persistence.

On implémente les classes DAO JDBC en nous basant sur les interfaces implémentées de base. On remplit chaque méthode en assurant la connexion avec la base de données ainsi que les requêtes SQL nécessaires. Ces implémentations sont directement visibles dans le dossier DAO.

Utilisez un PreparedStatement et des transactions lorsque nécessaire. Assurez-vous de gérer correctement la fermeture des ressources.

On utilise des objets PreparedStatement à chaque requête SQL utilisée.

2. Services

Adaptez la couche service pour qu'elle utilise les DAO JDBC au lieu des services en mémoire. Veillez à toujours respecter l'architecture du code !

On a adapté la couche service pour qu'elle utilise les DAO JDBC au lieu des services en mémoire. On a créé les services JDBC dans le dossier « jdbc », notamment JdbcArtistService.java, JdbcArtworkService.java, JdbcGalleryService.java, JdbcWorkshopService.java, JdbcCommunityService.java et JdbcExhibitionService.java.

Vous pouvez conserver une implémentation InMemoryXxxService pour les tests, mais l'application finale doit utiliser les services connectés à la base.

Les versions InMemory ont été conservées en commentaire dans ServiceProvider.java pour garder une possibilité de test, mais elles ne sont plus utilisées par l'application finale.

3. Connexion à la base

Complétez la classe de configuration DatabaseConfig (ou équivalent) pour fournir une connexion JDBC valide vers votre base ArtConnect.

On complète la configuration afin de fournir une connexion valide vers la base de données.

Paramétrez l'URL, l'utilisateur et le mot de passe dans un endroit clairement indiqué (fichier de configuration, constantes, etc.).

Le fichier DatabaseConfig.java centralise l'URL, l'utilisateur et le mot de passe. La connexion est fournie par ConnectionManager.java, qui charge explicitement le driver MySQL puis appelle DriverManager.getConnection(...).

4. Adaptation de l'UI

Sans modifier profondément l'interface graphique, assurez-vous que les actions principales (liste, ajout, modification, suppression) utilisent maintenant les services connectés à la base.

En termes d'interface graphique, on a rajouté un onglet « Registrations » pour pouvoir gérer les inscriptions des membres aux différents workshops/exhibitions. Cette page permet de :

- Sélectionner un membre parmi ceux dans la base
 - Voir les workshops/exhibitions auquel il est inscrit
- Sélectionner un workshop/exhibition
 - Voir les membres inscrit à ce dernier
- Inscrire/désinscrire le membre sélectionné au workshop/exhibition

De plus, on a rajouté trois sections à chaque page déjà existante :

1. Une première pour effectuer une recherche parmi les données de la table
2. Une deuxième pour ajouter une entrée à la table
3. Une dernière pour modifier ou supprimer l'entrée que l'utilisateur a sélectionné

Les contrôleurs utilisent maintenant les services connectés à la base. Par exemple, ArtistController.java, ArtworkController.java, ExhibitionController.java et WorkshopController.java récupèrent leurs données via ServiceProvider.java. Les données dans la base de données se mettent à jour à chaque opération sur l'application, qui sont donc répercutées sur l'affichage. (Par exemple un nouvel artiste est ajouté depuis l'application, cet artiste est rajouté dans la base de données, puis l'affichage est rafraîchi pour afficher ce nouvel ajout)

Vérifiez que les différents écrans affichent des données issues de votre base et que les opérations de CRUD sont persistantes.

Après avoir établi la connexion entre l'application et la base de données convenablement et avoir modifié ExhibitionController.java, les données issues de notre base s'affichent :

READ la table Artist

ArtConnect Pro - Local Art Community Platform

File Help

Discover Artists Artworks Galleries Exhibitions Workshops Community Registrations

▼ Search

Search artists...

▶ Add New Artist

▶ Update or Delete Artist

Artists List

Name	City	Email	Birth Year
Sophie Martin	Paris	sophie.martin@art.fr	1985
Lucas Bernard	Lyon	lucas.bernard@studio.fr	1978
Amira Hassan	Marseille	amira.hassan@gallery.eg	1992
Kenji Tanaka	Paris	kenji.tanaka@artmail.jp	1969
Elena Volkov	Bordeaux	elena.volkov@brush.ru	1990
Marc Dupont	Nantes	marc.dupont@photoart.fr	1975
Isabelle Noir	Toulouse	isabelle.noir@sculpt.fr	1983
Carlos Reyes	Paris	carlos.reyes@arte.mx	1988
Fatima Benali	Lille	fatima.benali@creation.ma	1995
Pierre Leroy	Strasbourg	pierre.leroy@gravure.fr	1971
Nadia Schmidt	Paris	nadia.schmidt@kunst.de	1980
Thomas Girard	Rennes	thomas.girard@digital.fr	1993
Yuki Nakamura	Nice	yuki.nakamura@washi.jp	1987
Omar Diallo	Paris	omar.diallo@pigment.sn	1976
Chloé Petit	Grenoble	chloe.petit@aquarelle.fr	1998

ArtConnect Pro v1.0 | Mode: In-Memory

1 • SELECT * FROM artconnect.artist;

Result Grid

	IdArtist	Email	Name	City	BirthYear
1		sophie.martin@art.fr	Sophie Martin	Paris	1985
2		lucas.bernard@studio.fr	Lucas Bernard	Lyon	1978
3		amira.hassan@gallery.eg	Amira Hassan	Marseille	1992
4		kenji.tanaka@artmail.jp	Kenji Tanaka	Paris	1969
5		elena.volkov@brush.ru	Elena Volkov	Bordeaux	1990
6		marc.dupont@photoart.fr	Marc Dupont	Nantes	1975
7		isabelle.noir@sculpt.fr	Isabelle Noir	Toulouse	1983
8		carlos.reyes@arte.mx	Carlos Reyes	Paris	1988
9		fatima.benali@creation.ma	Fatima Benali	Lille	1995
10		pierre.leroy@gravure.fr	Pierre Leroy	Strasbourg	1971
11		nadia.schmidt@kunst.de	Nadia Schmidt	Paris	1980
12		thomas.girard@digital.fr	Thomas Girard	Rennes	1993
13		yuki.nakamura@washi.jp	Yuki Nakamura	Nice	1987
14		omar.diallo@pigment.sn	Omar Diallo	Paris	1976
15		chloe.petit@aquarelle.fr	Chloé Petit	Grenoble	1998

CREATE

ArtConnect Pro - Local Art Community Platform

File Help

Discover Artists Artworks Galleries Exhibitions Workshops Community Registrations

▼ Search

Search artists...

▶ Add New Artist

▶ Update or Delete Artist

Artists List

Name	City	Email	Birth Year
Sophie Martin	Paris	sophie.martin@art.fr	1985
Lucas Bernard	Lyon	lucas.bernard@studio.fr	1978
Amira Hassan	Marseille	amira.hassan@gallery.eg	1992
Kenji Tanaka	Paris	kenji.tanaka@artmail.jp	1969
Elena Volkov	Bordeaux	elena.volkov@brush.ru	1990
Marc Dupont	Nantes	marc.dupont@photoart.fr	1975
Isabelle Noir	Toulouse	isabelle.noir@sculpt.fr	1983
Carlos Reyes	Paris	carlos.reyes@arte.mx	1988
Fatima Benali	Lille	fatima.benali@creation.ma	1995
Pierre Leroy	Strasbourg	pierre.leroy@gravure.fr	1971
Nadia Schmidt	Paris	nadia.schmidt@kunst.de	1980
Thomas Girard	Rennes	thomas.girard@digital.fr	1993
Yuki Nakamura	Nice	yuki.nakamura@washi.jp	1987
Omar Diallo	Paris	omar.diallo@pigment.sn	1976
Chloé Petit	Grenoble	chloe.petit@aquarelle.fr	1998
test	test	test	2000

ArtConnect Pro v1.0 | Mode: In-Memory

SQL File 3*

Limit to 1000 rows

1 • SELECT * FROM artist;

Result Grid

	IdArtist	Email	Name	City	BirthYear
3		amira.hassan@gallery.eg	Amira Hassan	Marseille	1992
4		kenji.tanaka@artmail.jp	Kenji Tanaka	Paris	1969
5		elena.volkov@brush.ru	Elena Volkov	Bordeaux	1990
6		marc.dupont@photoart.fr	Marc Dupont	Nantes	1975
7		isabelle.noir@sculpt.fr	Isabelle Noir	Toulouse	1983
8		carlos.reyes@arte.mx	Carlos Reyes	Paris	1988
9		fatima.benali@creation.ma	Fatima Benali	Lille	1995
10		pierre.leroy@gravure.fr	Pierre Leroy	Strasbourg	1971
11		nadia.schmidt@kunst.de	Nadia Schmidt	Paris	1980
12		thomas.girard@digital.fr	Thomas Girard	Rennes	1993
13		yuki.nakamura@washi.jp	Yuki Nakamura	Nice	1987
14		omar.diallo@pigment.sn	Omar Diallo	Paris	1976
15		chloe.petit@aquarelle.fr	Chloé Petit	Grenoble	1998
16		test	test	test	2000

UPDATE

ArtConnect Pro - Local Art Community Platform

File Help

Discover Artists Artworks Galleries Exhibitions Workshops Community Registrations

▼ Search

Search artists... Search Reset

► Add New Artist

► Update or Delete Artist

Artists List

Name	City	Email	Birth Year
Sophie Martin	Paris	sophie.martin@art.fr	1985
Lucas Bernard	Lyon	lucas.bernard@studio.fr	1978
Amira Hassan	Marseille	amira.hassan@gallery.eg	1992
Kenji Tanaka	Paris	kenji.tanaka@artmail.jp	1969
Elena Volkov	Bordeaux	elena.volkov@brush.ru	1990
Marc Dupont	Nantes	marc.dupont@photoart.fr	1975
Isabelle Noir	Toulouse	isabelle.noir@sculpt.fr	1983
Carlos Reyes	Paris	carlos.reyes@arte.mx	1988
Fatima Benali	Lille	fatima.benali@creation.ma	1995
Pierre Leroy	Strasbourg	pierre.leroy@gravure.fr	1971
Nadia Schmidt	Paris	nadia.schmidt@kunst.de	1980
Thomas Girard	Rennes	thomas.girard@digital.fr	1993
Yuki Nakamura	Nice	yuki.nakamura@wash.jp	1987
Omar Diallo	Paris	omar.diallo@pigment.sn	1976
Chloé Petit	Grenoble	chloe.petit@aquarelle.fr	1998
test_updated	test	test	2000

ArtConnect Pro v1.0 | Mode: In-Memory

SQL File 3*

Limit to 1000 rows

1 • SELECT * FROM artist;

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content: F5

IdArtist	Email	Name	City	BirthYear
3	amira.hassan@gallery.eg	Amira Hassan	Marseille	1992
4	kenji.tanaka@artmail.jp	Kenji Tanaka	Paris	1969
5	elena.volkov@brush.ru	Elena Volkov	Bordeaux	1990
6	marc.dupont@photoart.fr	Marc Dupont	Nantes	1975
7	isabelle.noir@sculpt.fr	Isabelle Noir	Toulouse	1983
8	carlos.reyes@arte.mx	Carlos Reyes	Paris	1988
9	fatima.benali@creation.ma	Fatima Benali	Lille	1995
10	pierre.leroy@gravure.fr	Pierre Leroy	Strasbourg	1971
11	nadia.schmidt@kunst.de	Nadia Schmidt	Paris	1980
12	thomas.girard@digital.fr	Thomas Girard	Rennes	1993
13	yuki.nakamura@wash.jp	Yuki Nakamura	Nice	1987
14	omar.diallo@pigment.sn	Omar Diallo	Paris	1976
15	chloe.petit@aquarelle.fr	Chloé Petit	Grenoble	1998
16	test	test_updated	test	2000

DELETE

ArtConnect Pro - Local Art Community Platform

File Help

Discover Artists Artworks Galleries Exhibitions Workshops Community Registrations

▼ Search

Search artists... Search Reset

► Add New Artist

► Update or Delete Artist

Artists List

Name	City	Email	Birth Year
Sophie Martin	Paris	sophie.martin@art.fr	1985
Lucas Bernard	Lyon	lucas.bernard@studio.fr	1978
Amira Hassan	Marseille	amira.hassan@gallery.eg	1992
Kenji Tanaka	Paris	kenji.tanaka@artmail.jp	1969
Elena Volkov	Bordeaux	elena.volkov@brush.ru	1990
Marc Dupont	Nantes	marc.dupont@photoart.fr	1975
Isabelle Noir	Toulouse	isabelle.noir@sculpt.fr	1983
Carlos Reyes	Paris	carlos.reyes@arte.mx	1988
Fatima Benali	Lille	fatima.benali@creation.ma	1995
Pierre Leroy	Strasbourg	pierre.leroy@gravure.fr	1971
Nadia Schmidt	Paris	nadia.schmidt@kunst.de	1980
Thomas Girard	Rennes	thomas.girard@digital.fr	1993
Yuki Nakamura	Nice	yuki.nakamura@wash.jp	1987
Omar Diallo	Paris	omar.diallo@pigment.sn	1976
Chloé Petit	Grenoble	chloe.petit@aquarelle.fr	1998

ArtConnect Pro v1.0 | Mode: In-Memory

SQL File 3*

Limit to 1000 rows

1 • SELECT * FROM artist;

Result Grid Filter Rows: Edit: Export/Import: Wrap Cell Content: F5

IdArtist	Email	Name	City	BirthYear
1	sophie.martin@art.fr	Sophie Martin	Paris	1985
2	lucas.bernard@studio.fr	Lucas Bernard	Lyon	1978
3	amira.hassan@gallery.eg	Amira Hassan	Marseille	1992
4	kenji.tanaka@artmail.jp	Kenji Tanaka	Paris	1969
5	elena.volkov@brush.ru	Elena Volkov	Bordeaux	1990
6	marc.dupont@photoart.fr	Marc Dupont	Nantes	1975
7	isabelle.noir@sculpt.fr	Isabelle Noir	Toulouse	1983
8	carlos.reyes@arte.mx	Carlos Reyes	Paris	1988
9	fatima.benali@creation.ma	Fatima Benali	Lille	1995
10	pierre.leroy@gravure.fr	Pierre Leroy	Strasbourg	1971
11	nadia.schmidt@kunst.de	Nadia Schmidt	Paris	1980
12	thomas.girard@digital.fr	Thomas Girard	Rennes	1993
13	yuki.nakamura@wash.jp	Yuki Nakamura	Nice	1987
14	omar.diallo@pigment.sn	Omar Diallo	Paris	1976
15	chloe.petit@aquarelle.fr	Chloé Petit	Grenoble	1998